V2X PRIME API Rev 0.1

NXP Copyright

Generated by Doxygen 1.9.1

1 PRIME API 1

2 Revision History 3

3 General concepts related to the API 5

3.1 Basic Concept . . . 5
3.2 Session . . . 5
3.3 PRIME Flow . . . 6

4 User space MU driver 7

5 ATU with PRIME 9

5.1 RX Path (Hardware → Memory → CPU) . . . 9
5.2 TX Path (CPU → Memory → Hardware) . . . 10
5.3 ATU Configuration . . . 10
5.4 Data Flow for User Allocated Memory with ATU . . . 10
5.5 Data Flow for SE Allocated Memory with ATU . . . 11

6 Module Index 13

6.1 Modules . . . 13

7 Module Documentation 15

7.1 PRIME AES Cryptographic Operations . . . 15
7.1.1 Detailed Description . . . 16
7.1.2 Data Structure Documentation . . . 16
7.1.2.1 struct aes_key_t . . . 16
7.1.2.2 struct aes_op_params . . . 16
7.1.2.3 struct aead_op_params . . . 16
7.1.3 Enumeration Type Documentation . . . 17
7.1.3.1 crypto_cipher_algorithm . . . 17
7.1.3.2 crypto_aead_algorithm . . . 17
7.1.3.3 crypto_cipher_operation . . . 18
7.1.4 Function Documentation . . . 18
7.1.4.1 prime_cipher_init() . . . 18
7.2 PRIME Handle Management . . . 18
7.2.1 Detailed Description . . . 19
7.2.2 Macro Definition Documentation . . . 19
7.2.2.1 PRIME_HANDLE_NONE . . . 19
7.2.3 Typedef Documentation . . . 19
7.2.3.1 prime_hdl_t . . . 19
7.3 PRIME HMAC and Hash Operations . . . 19
7.3.1 Detailed Description . . . 20
7.3.2 Data Structure Documentation . . . 20
7.3.2.1 struct hmac_key_t . . . 20

7.3.2.2 struct mac_op_params . . . 21
7.3.2.3 struct hash_one_go_args_t . . . 21
7.3.2.4 struct hash_one_go_op_params . . . 21
7.3.3 Enumeration Type Documentation . . . 22
7.3.3.1 crypto_mac_operation . . . 22
7.3.3.2 crypto_sha_algorithm . . . 22
7.3.3.3 crypto_hash_algorithm . . . 22
7.3.4 Function Documentation . . . 23
7.3.4.1 prime_mac_init() . . . 23
7.3.4.2 prime_hash_one_go() . . . 23
7.4 PRIME Information Retrieval . . . 24
7.4.1 Detailed Description . . . 24
7.4.2 Data Structure Documentation . . . 24
7.4.2.1 struct prime_info_t . . . 24
7.4.3 Function Documentation . . . 25
7.4.3.1 prime_get_info() . . . 25
7.5 PRIME Service Management . . . 25
7.5.1 Detailed Description . . . 26
7.5.2 Data Structure Documentation . . . 26
7.5.2.1 struct open_service_args_t . . . 26
7.5.3 Macro Definition Documentation . . . 27
7.5.3.1 MAX_PRIME_KEY_SLOT_SUPPORTED . . . 27
7.5.4 Enumeration Type Documentation . . . 27
7.5.4.1 service_type . . . 27
7.5.5 Function Documentation . . . 27
7.5.5.1 prime_open_service() . . . 28
7.5.5.2 prime_close_service() . . . 28
7.6 PRIME Utility Definitions . . . 28
7.6.1 Detailed Description . . . 29
7.6.2 Data Structure Documentation . . . 29
7.6.2.1 struct buf . . . 29
7.6.3 Enumeration Type Documentation . . . 29
7.6.3.1 prime_err_t . . . 29
7.7 V2X Version Information . . . 30
7.7.1 Detailed Description . . . 30
7.7.2 Data Structure Documentation . . . 30
7.7.2.1 struct v2x_version_t . . . 30
7.7.3 Function Documentation . . . 31
7.7.3.1 get_v2x_version() . . . 31
7.8 PRIME Cryptographic Engine . . . 31
7.8.1 Detailed Description . . . 32
7.8.2 Data Structure Documentation . . . 32

7.8.2.1 struct status_buf . . . 32
7.8.2.2 struct crypto_status . . . 33
7.8.2.3 struct prime_crypto_op . . . 33
7.8.2.4 union prime_crypto_op.__unnamed2__ . . . 33
7.8.3 Typedef Documentation . . . 34
7.8.3.1 status_buf_t . . . 34
7.8.3.2 crypto_op_args_t . . . 34
7.8.4 Enumeration Type Documentation . . . 34
7.8.4.1 crypto_op_type . . . 34
7.8.4.2 crypto_op_status . . . 35
7.8.5 Function Documentation . . . 35
7.8.5.1 prime_process_ops() . . . 35

Index **37**

Chapter 1

PRIME API

This document is a software reference description of the API provided by the Secure Enclave Library on i.MX937, i.MX941, i.MX942, i.MX943 and i.MX952 platforms for V2X PRIME solutions

Chapter 2

Revision History

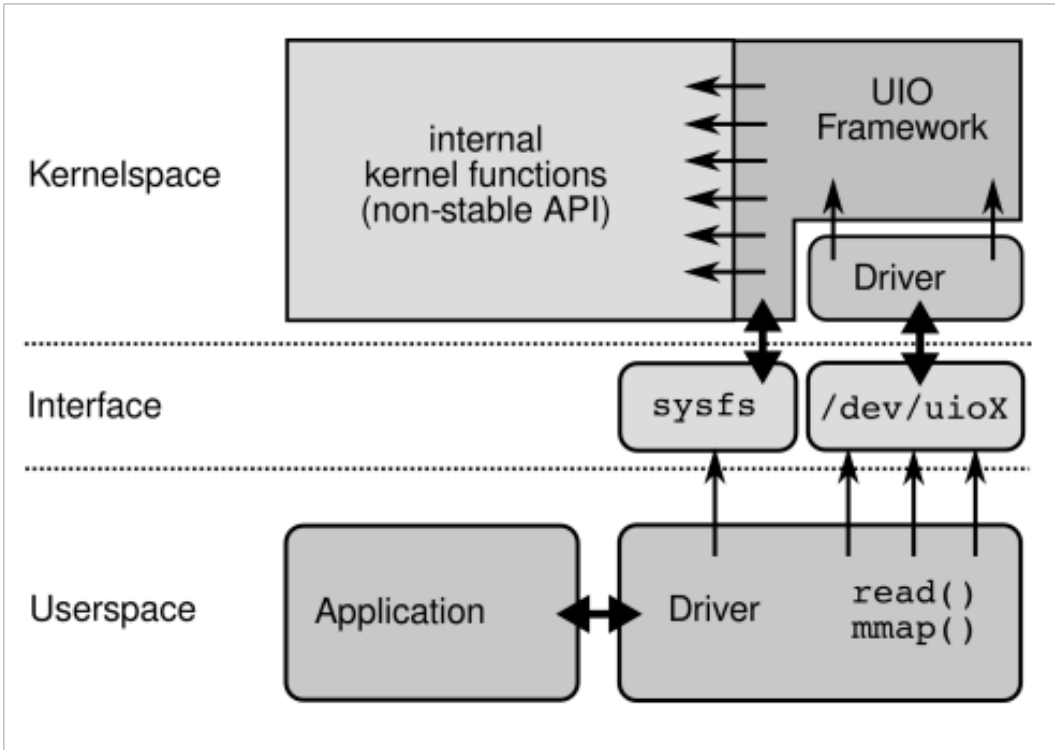
Revision	date	description
0.1	May 29 2026	Initial Draft
0.2	July 14 2026	Updated supported platforms list

Chapter 3

General concepts related to the API

3.1 Basic Concept

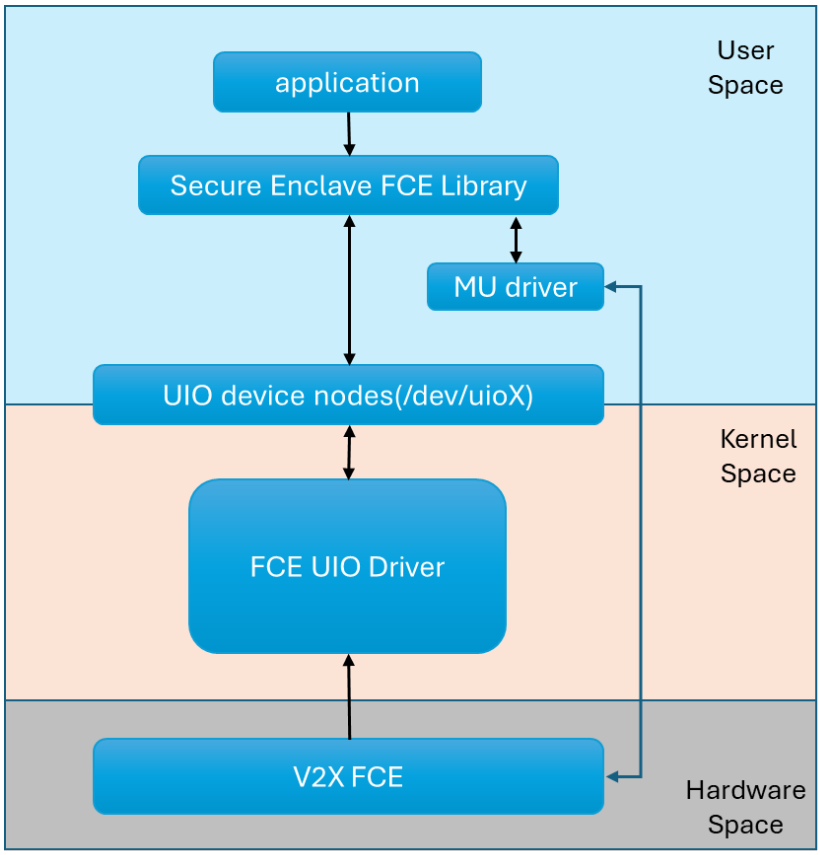
The UIO framework generates a set of directories and attribute files in sysfs. These sysfs attributes allow searching for devices, finding the correct size of the memory mappings.



3.2 Session

The API must be initialized by a potential requestor by opening a session. The session establishes a route between the requester and the PRIME. When a session is opened, the PRIME returns a handle identifying the session to the requester.

3.3 PRIME Flow



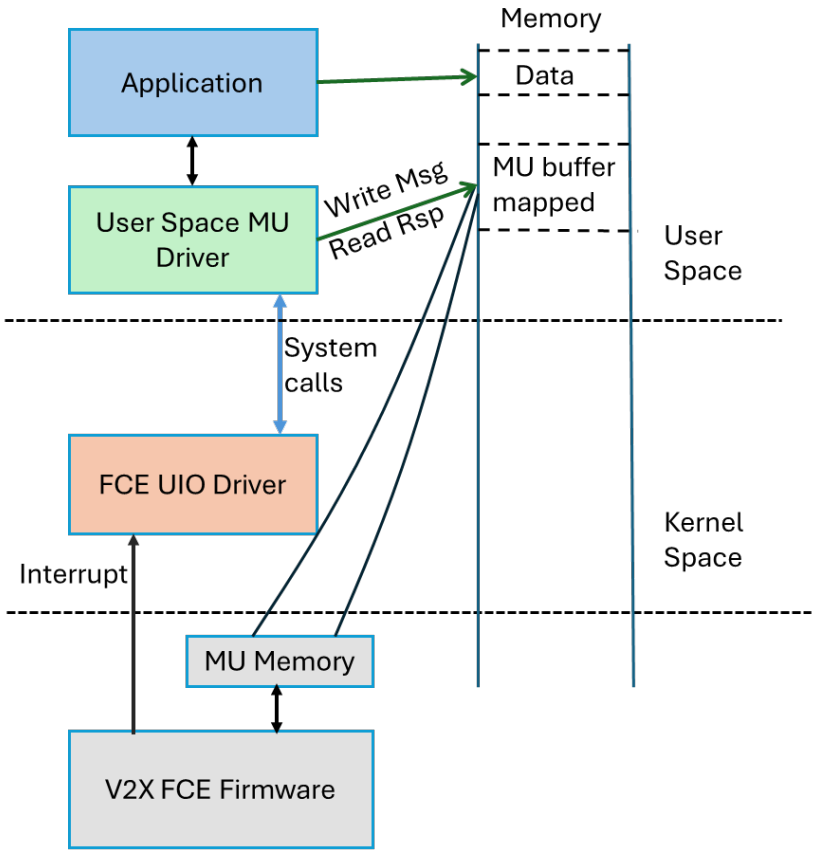
UIO Flow

1. Initialization
   - The PRIME library initializes the PRIME hardware.
   - The library sets up the Mailbox driver.
2. Request Submission
   - Cryptographic operations are defined using PRIME Requests.
   - PRIME requests specify the operations to be performed, input data, and output buffers.
   - Requests are submitted to the PRIME hardware via mailbox.
   - If any error occurs during parameter validation or while pushing the requests, then the complete operation will be nacked with error.
3. Processing
   - The PRIME hardware reads the request and processes the cryptographic operations.
   - It uses DMA engines to read input data and write results to output buffers.
   - The hardware performs the specified cryptographic operations (e.g., encryption, hashing).
4. Completion
   - Once the operations are completed, the results are written to the output buffers.
   - The driver retrieves the results and notifies the application of job completion.
   - Job completion result is shared in the u32_t status_buffer, provided by the user itself.
   - In case of success, status_buffer will have (0xD6) and in case of failure (0x29).

Chapter 4

User space MU driver

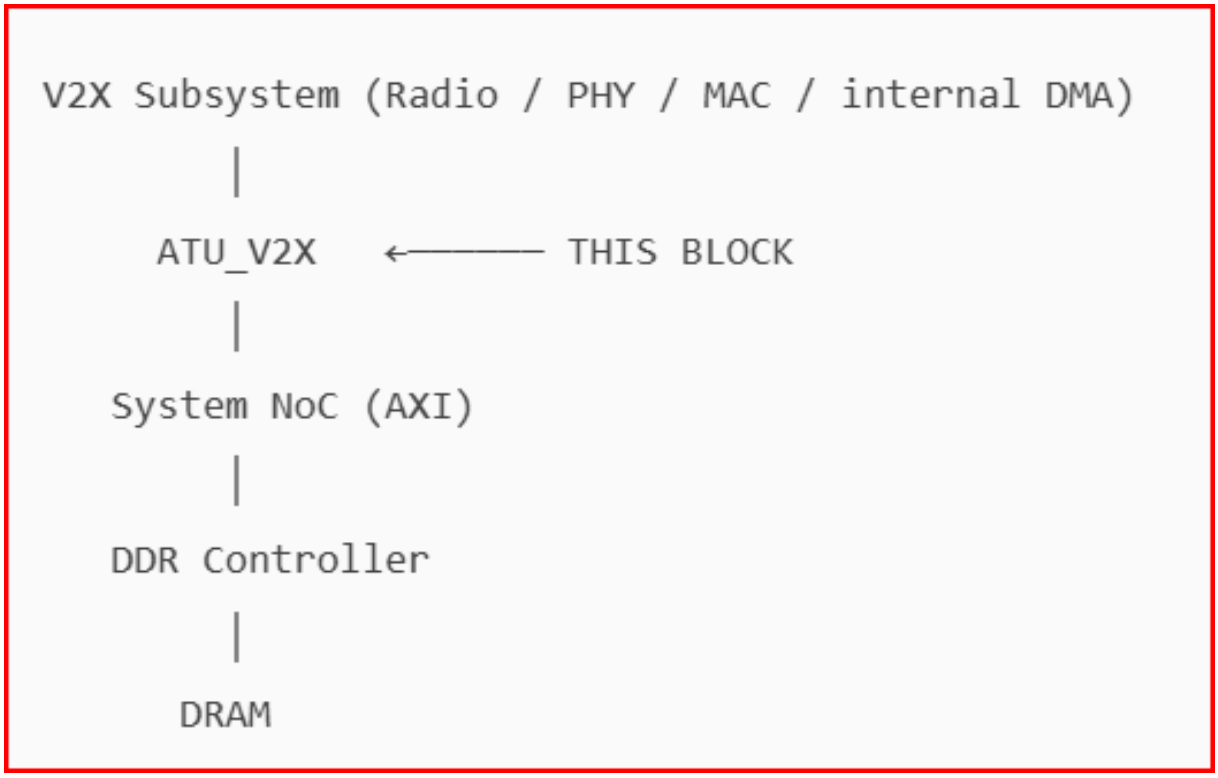
User space application opens the UIO device file and mmap the Mailbox memory region directly to user-space memory. This allows the user-space application to directly read/write the message from user space. Data buffers can be managed and accessed directly by the application without overhead of a copy.



User Space Communication over mapped mailbox memory

Chapter 5

ATU with PRIME



ATU_V2X (Address Translation Unit for V2X) is a hardware block that maps the V2X subsystem's local DMA address space to SoC physical DRAM.

5.1 RX Path (Hardware → Memory → CPU)

1. V2X PHY receives data
2. V2X DMA writes to local address 0x0002_0000
3. ATU_V2X translates to DRAM address 0x9002_0000
4. DDR write occurs
5. CPU later reads buffer

5.2 TX Path (CPU → Memory → Hardware)

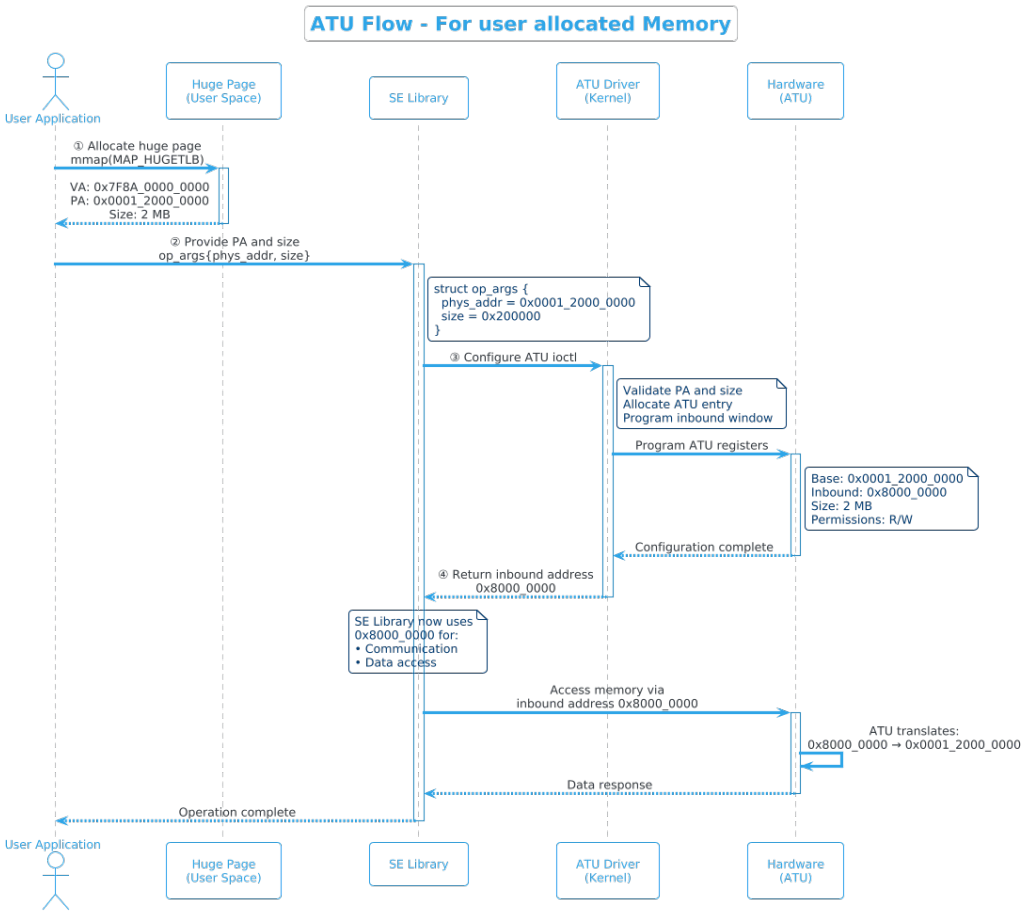
1. CPU writes packet to DRAM buffer
2. Clean cache
3. V2X DMA reads local address 0x0003_0000
4. ATU_V2X translates to DRAM
5. Packet transmitted

5.3 ATU Configuration

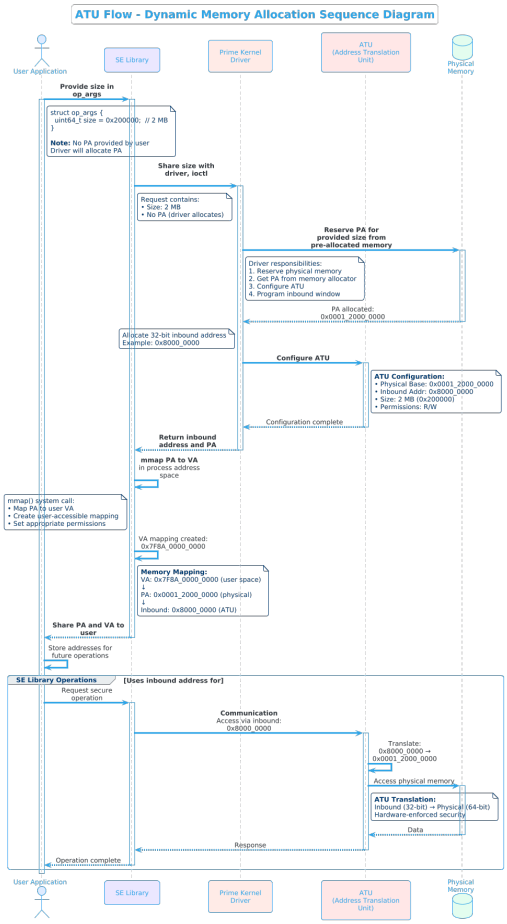
ATU_V2X never touches caches. A hybrid approach is used to configure ATU:

1. User can provide its own physical address and size to configure ATU (maximum 1 GB supported) - not recommended.
2. User can request SE library to allocate physical address and size (maximum 1 GB supported) - recommended approach.

5.4 Data Flow for User Allocated Memory with ATU



5.5 Data Flow for SE Allocated Memory with ATU



Chapter 6

Module Index

6.1 Modules

Here is a list of all modules:

PRIME AES Cryptographic Operations 15
PRIME HMAC and Hash Operations 19
PRIME Cryptographic Engine 31
- PRIME Handle Management 18
- PRIME Information Retrieval 24
- PRIME Service Management 25
- PRIME Utility Definitions 28
- V2X Version Information 30

Chapter 7

Module Documentation

7.1 PRIME AES Cryptographic Operations

AES cipher and AEAD operations for PRIME security services.

Data Structures

- struct aes_key_t

  Structure describing the AES key configuration. *More...*
- struct aes_op_params

  AES cipher operation parameters. *More...*
- struct aead_op_params

  AES AEAD operation parameters. *More...*

Enumerations

- enum crypto_cipher_algorithm {
  CRYPTO_CIPHER_AES_CBC = 1 ,
  CRYPTO_CIPHER_AES_ECB ,
  CRYPTO_CIPHER_AES_CTR }

  AES cipher algorithm modes.
- enum crypto_aead_algorithm {
  CRYPTO_AEAD_AES_GCM = 4 ,
  CRYPTO_AEAD_AES_CCM }

  AES AEAD (Authenticated Encryption with Associated Data) algorithm modes.
- enum crypto_cipher_operation {
  CRYPTO_CIPHER_OP_DECRYPT ,
  CRYPTO_CIPHER_OP_ENCRYPT }

  Cipher operation direction.

Functions

- prime_err_t prime_cipher_init (prime_hdl_t service_hdl, aes_key_t *args)

  Initialize AES cipher with the specified key.

7.1.1 Detailed Description

AES cipher and AEAD operations for PRIME security services.

This module provides AES encryption/decryption capabilities including:

- AES cipher modes (CBC, ECB, CTR)
- AEAD modes (GCM, CCM)
- Key management and initialization

7.1.2 Data Structure Documentation

7.1.2.1 struct aes_key_t

Structure describing the AES key configuration.

This structure is used to configure AES keys for cryptographic operations. Keys can be provided directly or referenced via keyslot.

Data Fields

const uint8_t *	key	Pointer to the key buffer.
uint32_t	keylen	Length of the key in bytes (16, 24, or 32 for AES-128/192/256)
uint8_t	keyslot	Key slot identifier for hardware key storage, valid values 0...63. SE library will allocate the available key_slot and return the value in keyslot

7.1.2.2 struct aes_op_params

AES cipher operation parameters.

Structure containing all parameters required for AES cipher operations (CBC, ECB, CTR modes).

Data Fields

enum crypto_cipher_algorithm	algo	Cipher algorithm to use.
uint8_t	keyslot	Key slot identifier.
uint8_t	enc	Operation direction (0=decrypt, 1=encrypt)
uint8_t *	iv	Pointer to initialization vector buffer.
uint16_t	ivlen	Length of initialization vector in bytes.

7.1.2.3 struct aead_op_params

AES AEAD operation parameters.

Structure containing all parameters required for AEAD operations (GCM, CCM modes) including additional authenticated data and tag.

Data Fields

enum crypto_aead_algorithm	algo	AEAD algorithm to use.
uint8_t	keyslot	Key slot identifier.
uint8_t	enc	Operation direction (0=decrypt, 1=encrypt)
uint8_t *	iv	Pointer to initialization vector/nonce buffer.
uint16_t	ivlen	Length of initialization vector in bytes.
struct buf	aad	Additional Authenticated Data buffer. Known Limitation : This cannot be NULL. Prime expects a valid address, else PUSH will fail
struct buf	tag	Authentication tag buffer.

7.1.3 Enumeration Type Documentation

7.1.3.1 crypto_cipher_algorithm

`enum crypto_cipher_algorithm`

AES cipher algorithm modes.

Supported AES cipher modes for encryption and decryption operations.

Enumerator

CRYPTO_CIPHER_AES_CBC	AES Cipher Block Chaining mode.
CRYPTO_CIPHER_AES_ECB	AES Electronic Codebook mode.
CRYPTO_CIPHER_AES_CTR	AES Counter mode.

7.1.3.2 crypto_aead_algorithm

`enum crypto_aead_algorithm`

AES AEAD (Authenticated Encryption with Associated Data) algorithm modes.

Supported AEAD modes that provide both confidentiality and authenticity.

Enumerator

CRYPTO_AEAD_AES_GCM	AES Galois/Counter Mode.
CRYPTO_AEAD_AES_CCM	AES Counter with CBC-MAC.

7.1.3.3 crypto_cipher_operation

```
enum crypto_cipher_operation
```

Cipher operation direction.

Specifies whether the operation is encryption or decryption.

Enumerator

CRYPTO_CIPHER_OP_DECRYPT	Decryption operation.
CRYPTO_CIPHER_OP_ENCRYPT	Encryption operation.

7.1.4 Function Documentation

7.1.4.1 prime_cipher_init()

```
prime_err_t prime_cipher_init (
            prime_hdl_t service_hdl,
            aes_key_t * args )
```

Initialize AES cipher with the specified key.

This function initializes the AES cipher engine with the provided key configuration. The key can be loaded into a hardware keyslot for subsequent cryptographic operations.

Parameters

in	*service_hdl*	Handle to the PRIME service session
in	*args*	Pointer to AES key configuration structure

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if parameters are invalid

PRIME_ERR_SERVICE_OPEN if service is not properly opened

PRIME_ERROR on other errors

7.2 PRIME Handle Management

Handle definitions and management for PRIME services.

Macros

- #define PRIME_HANDLE_NONE (0x0)

  Invalid/uninitialized handle value.

Typedefs

- typedef uint32_t prime_hdl_t

  PRIME handle type definition.

7.2.1 Detailed Description

Handle definitions and management for PRIME services.

This module provides handle type definitions used across PRIME services for managing service sessions and resources.

7.2.2 Macro Definition Documentation

7.2.2.1 PRIME_HANDLE_NONE

```
#define PRIME_HANDLE_NONE (0x0)
```

Invalid/uninitialized handle value.

This value represents an invalid or uninitialized PRIME handle. Used to indicate that no valid service handle has been assigned.

7.2.3 Typedef Documentation

7.2.3.1 prime_hdl_t

```
typedef uint32_t prime_hdl_t
```

PRIME handle type definition.

Handle used to reference PRIME service sessions. Each opened service is assigned a unique handle that must be used for all subsequent operations on that service.

7.3 PRIME HMAC and Hash Operations

HMAC generation/verification and SHA hash operations for PRIME security services.

Data Structures

- struct hmac_key_t

  Structure describing the HMAC key configuration. More...
- struct mac_op_params

  MAC operation parameters. More...
- struct hash_one_go_args_t

  Hash one go arguments passed to prime_hash_one_go() More...
- struct hash_one_go_op_params

  Hash one go operation parameters. More...

Enumerations

- enum crypto_mac_operation {
  CRYPTO_AUTH_OP_HMAC_GEN ,
  CRYPTO_AUTH_OP_HMAC_VERIFY ,
  CRYPTO_AUTH_OP_SHA_GEN }

  MAC operation types.
- enum crypto_sha_algorithm {
  CRYPTO_AUTH_SHA_256 = 1 ,
  CRYPTO_AUTH_SHA_384 ,
  CRYPTO_AUTH_SHA_512 }

  SHA hash algorithm types.
- enum crypto_hash_algorithm {
  CRYPTO_HASH_SHA2_256 = 0x01 ,
  CRYPTO_HASH_SHA2_384 = 0x02 ,
  CRYPTO_HASH_SHA2_512 = 0x03 }

  FCE Hash algorithm identifiers.

Functions

- prime_err_t prime_mac_init (prime_hdl_t service_hdl, hmac_key_t *args)

  Initialize HMAC with the specified key.
- prime_err_t prime_hash_one_go (prime_hdl_t service_hdl, hash_one_go_args_t *args)

  Perform a hash one go operation.

7.3.1 Detailed Description

HMAC generation/verification and SHA hash operations for PRIME security services.

This module provides message authentication and hashing capabilities including:

- HMAC generation and verification
- SHA-256, SHA-384, SHA-512 hash operations
- Key management for HMAC operations

7.3.2 Data Structure Documentation

7.3.2.1 struct hmac_key_t

Structure describing the HMAC key configuration.

This structure is used to configure HMAC keys for message authentication operations. Keys can be provided directly or referenced via keyslot.

Data Fields

const uint8_t *	key	Pointer to the HMAC key buffer.
uint32_t	keylen	Length of the key in bytes.
uint8_t	keyslot	Key slot identifier for hardware key storage, valid value 0...63. SE library will allocate the available key_slot and return the value in keyslot
uint8_t	sha_algo	SHA algorithm to use with HMAC (from crypto_sha_algorithm enum)

7.3.2.2 struct mac_op_params

MAC operation parameters.

Structure containing all parameters required for HMAC generation, verification, or SHA hash operations.

Data Fields

enum crypto_sha_algorithm	sha_algo	SHA algorithm to use for the operation.
uint8_t	keyslot	Key slot identifier (for HMAC operations)
uint8_t	dir	Operation direction/type (from crypto_mac_operation enum)
struct buf	digest	Buffer for digest/MAC output or verification input.

7.3.2.3 struct hash_one_go_args_t

Hash one go arguments passed to prime_hash_one_go()

Data Fields

enum crypto_hash_algorithm	algo	Hash algorithm to use.
uint8_t	flags	Reserved for future use; must be set to 0, currently ignored.
const uint8_t *	input	Pointer to input data buffer (virtual address)
uint32_t	input_size	Length in bytes of the input payload.
uint8_t *	output	Pointer to output digest buffer (virtual address) must have ALIGN_UP(output_size, 64) + sizeof(status_buf_t) bytes allocated
uint32_t	output_size	Length in bytes of the output digest.
uint64_t	input_phys	Physical address of input buffer (DMA accessible, 64-byte aligned)
uint64_t	output_phys	Physical address of output buffer (DMA accessible, 64-byte aligned)

7.3.2.4 struct hash_one_go_op_params

Hash one go operation parameters.

Parameters for a standalone hash operation that does not require a pre-loaded key. The caller provides the algorithm to use and a buffer to receive the output digest.

Data Fields

enum crypto_hash_algorithm	algo	Hash algorithm to use.
uint8_t	flags	Reserved for future use; must be set to 0, currently ignored.
struct buf	digest	Output digest buffer (virt + phys addresses, len = digest size)

7.3.3 Enumeration Type Documentation

7.3.3.1 crypto_mac_operation

`enum crypto_mac_operation`

MAC operation types.

Supported message authentication code operations including HMAC generation, verification, and standalone SHA hash generation.

Enumerator

CRYPTO_AUTH_OP_HMAC_GEN	HMAC generation operation.
CRYPTO_AUTH_OP_HMAC_VERIFY	HMAC verification operation.
CRYPTO_AUTH_OP_SHA_GEN	SHA hash generation operation (without HMAC)

7.3.3.2 crypto_sha_algorithm

`enum crypto_sha_algorithm`

SHA hash algorithm types.

Supported SHA (Secure Hash Algorithm) variants for hashing and HMAC operations.

Enumerator

CRYPTO_AUTH_SHA_256	SHA-256 algorithm (256-bit digest)
CRYPTO_AUTH_SHA_384	SHA-384 algorithm (384-bit digest)
CRYPTO_AUTH_SHA_512	SHA-512 algorithm (512-bit digest)

7.3.3.3 crypto_hash_algorithm

`enum crypto_hash_algorithm`

FCE Hash algorithm identifiers.

These values are placed in bits[5:4] of the FCE flags word (Word 2) of the HASH Generate command (0x80): 1 = SHA256, 2 = SHA384, 3 = SHA512

Enumerator

CRYPTO_HASH_SHA2_256	SHA2-256 (32-byte digest) - FCE flags bits[5:4] = 1.
CRYPTO_HASH_SHA2_384	SHA2-384 (48-byte digest) - FCE flags bits[5:4] = 2.
CRYPTO_HASH_SHA2_512	SHA2-512 (64-byte digest) - FCE flags bits[5:4] = 3.

7.3.4 Function Documentation

7.3.4.1 prime_mac_init()

```
prime_err_t prime_mac_init (
            prime_hdl_t service_hdl,
            hmac_key_t * args )
```

Initialize HMAC with the specified key.

This function initializes the HMAC engine with the provided key configuration. The key can be loaded into a hardware keyslot for subsequent HMAC operations.

Parameters

in	*service_hdl*	Handle to the PRIME service session
in	*args*	Pointer to HMAC key configuration structure

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if parameters are invalid

PRIME_ERR_SERVICE_OPEN if service is not properly opened

PRIME_ERROR on other errors

7.3.4.2 prime_hash_one_go()

```
prime_err_t prime_hash_one_go (
            prime_hdl_t service_hdl,
            hash_one_go_args_t * args )
```

Perform a hash one go operation.

Computes a hash digest over the input buffer in a single FCE operation.

Parameters

in	*service_hdl*	Handle to the PRIME service session
in	*args*	Pointer to hash one go arguments

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if parameters are invalid or NULL

PRIME_ERROR on other errors

7.4 PRIME Information Retrieval

PRIME service information and capabilities.

Data Structures

- struct prime_info_t

  PRIME service information structure. More...

Functions

- prime_err_t prime_get_info (prime_hdl_t serv_hdl, prime_info_t *prime_info)

  Get PRIME service information.

7.4.1 Detailed Description

PRIME service information and capabilities.

This module provides functionality to retrieve PRIME service information including firmware version and hardware capabilities.

7.4.2 Data Structure Documentation

7.4.2.1 struct prime_info_t

PRIME service information structure.

Structure containing PRIME firmware version and hardware capability information for an opened service session.

Data Fields

uint8_t	major	Major version number of PRIME firmware.
uint8_t	minor	Minor version number of PRIME firmware.
uint8_t	patch	Patch version number of PRIME firmware.
uint32_t	num_fifo_entries	Number of FIFO entries supported by hardware (queue depth)

7.4.3 Function Documentation

7.4.3.1 prime_get_info()

```
prime_err_t prime_get_info (
            prime_hdl_t serv_hdl,
            prime_info_t * prime_info )
```

Get PRIME service information.

Retrieves information about the PRIME service including firmware version and hardware capabilities such as FIFO depth. This information can be used to determine feature availability and optimize operation batching.

Parameters

in	*serv_hdl*	Handle to the PRIME service session
out	*prime_info*	Pointer to structure to be filled with service information

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if parameters are invalid or NULL

PRIME_ERR_SERVICE_OPEN if service handle is invalid

PRIME_ERROR on other errors

7.5 PRIME Service Management

PRIME service session lifecycle management.

Data Structures

- struct open_service_args_t

  Structure containing all parameters required to open a PRIME service session

. *More...*

Macros

- #define MAX_PRIME_KEY_SLOT_SUPPORTED 64

  Maximum Key slots supported by Prime.

Enumerations

- enum service_type {
  SERVICE_TYPE_NONE ,
  SERVICE_TYPE_ALL = 7 }

  PRIME service types.

Functions

- prime_err_t prime_open_service (open_service_args_t *op_serv_args, prime_hdl_t *service_hdl)

  Open a PRIME service session.
- void prime_close_service (prime_hdl_t service_hdl)

  Close a PRIME service session.

7.5.1 Detailed Description

PRIME service session lifecycle management.

This module provides functionality for opening and closing PRIME service sessions, managing shared memory buffers, and configuring service types.

7.5.2 Data Structure Documentation

7.5.2.1 struct open_service_args_t

Structure containing all parameters required to open a PRIME service session

.

Data Fields

prime_hdl_t	service_hdl	Service handle (output parameter, set by prime_open_service)
enum service_type	serv_type	Service type to open (determines available functionality)
uint64_t	physical_addr	Physical memory address for shared buffer (DMA accessible) if NULL, the SE library will provide the physical address to the user if not NULL, the SE library will program ATU(if 36 bit address), User needs to set the physical address for both 36 bit and 32 bit address range.
uint64_t	virtual_addr	Virtual memory address for shared buffer (CPU accessible) if physical address NULL, then SE library will provide the virtual address.
uint32_t	size	Size of shared buffer in bytes.

7.5.3 Macro Definition Documentation

7.5.3.1 MAX_PRIME_KEY_SLOT_SUPPORTED

```
#define MAX_PRIME_KEY_SLOT_SUPPORTED 64
```

Maximum Key slots supported by Prime.

This value represents the maximum number of key slots supported by prime firmware i.e. maximum number of keys that can be loaded at one time.

7.5.4 Enumeration Type Documentation

7.5.4.1 service_type

```
enum service_type
```

PRIME service types.

Enumerator

SERVICE_TYPE_NONE	No service type specified.
SERVICE_TYPE_ALL	All available services (full functionality)

7.5.5 Function Documentation

7.5.5.1 prime_open_service()

```
prime_err_t prime_open_service (
            open_service_args_t * op_serv_args,
            prime_hdl_t * service_hdl )
```

Open a PRIME service session.

Opens a new PRIME service session with the specified configuration. Allocates resources and establishes communication with the PRIME firmware through the Message Unit (MU). The shared memory buffer is used for exchanging cryptographic operation data.

Parameters

in,out	*op_serv_args*	Pointer to service open arguments structure
out	*service_hdl*	Pointer to receive the allocated service handle

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if parameters are invalid or NULL

PRIME_ERR_SERVICE_OPEN if service cannot be opened

PRIME_ERR_MEMORY_ALLOC if memory allocation fails

PRIME_ERROR on other errors

7.5.5.2 prime_close_service()

```
void prime_close_service (
            prime_hdl_t service_hdl )
```

Close a PRIME service session.

Closes an opened PRIME service session and releases all associated resources. All pending operations should be completed before closing the service.

Parameters

in	*service_hdl*	Handle to the service session to close

7.6 PRIME Utility Definitions

Common utility types and error codes for PRIME API.

Data Structures

- struct buf

  Buffer descriptor structure. More...

Enumerations

- enum prime_err_t {
  PRIME_ERR_NONE = 0 ,
  PRIME_ERR_INVALID_PARAM = 1 ,
  PRIME_ERR_MEMORY_ALLOC = 2 ,
  PRIME_ERR_SERVICE_OPEN = 3 ,
  PRIME_ERR_SERVICE_CLOSE = 4 ,
  PRIME_ERROR = 5 }

  PRIME error codes.

7.6.1 Detailed Description

Common utility types and error codes for PRIME API.

This module provides common definitions used across the PRIME API including error codes and buffer structures.

7.6.2 Data Structure Documentation

7.6.2.1 struct buf

Buffer descriptor structure.

Structure describing a memory buffer with both virtual and physical addresses. Used for DMA operations where hardware needs physical addresses and software needs virtual addresses.

Data Fields

uint8_t *	virt_addr	Virtual address of buffer (CPU accessible)
uint64_t	phys_addr	Physical address of buffer (DMA/hardware accessible) This address needs to be 64 byte aligned because Cache-line alignment is 64 bytes
uint32_t	len	Length of buffer in bytes.

7.6.3 Enumeration Type Documentation

7.6.3.1 prime_err_t

```
enum prime_err_t
```

PRIME error codes.

Enumeration of error codes returned by PRIME API functions. All PRIME functions return these standardized error codes to indicate success or failure reasons.

Enumerator

PRIME_ERR_NONE	Operation completed successfully.
PRIME_ERR_INVALID_PARAM	Invalid parameter provided (NULL pointer, out of range value, etc.)
PRIME_ERR_MEMORY_ALLOC	Memory allocation failed.
PRIME_ERR_SERVICE_OPEN	Service open operation failed.
PRIME_ERR_SERVICE_CLOSE	Service close operation failed.
PRIME_ERROR	Generic error (unspecified failure)

7.7 V2X Version Information

V2X software version retrieval.

Data Structures

- struct v2x_version_t

  V2X version information structure. More...

Functions

- prime_err_t get_v2x_version (v2x_version_t *args)

  Get V2X software version information.

7.7.1 Detailed Description

V2X software version retrieval.

This module provides functionality to retrieve V2X software version information including major, minor, patch numbers and commit ID.

7.7.2 Data Structure Documentation

7.7.2.1 struct v2x_version_t

V2X version information structure.

Structure containing version information for the V2X software stack. Follows semantic versioning (major.minor.↩ patch) with additional commit identifier.

Data Fields

uint8_t	major	Major version number (incremented for incompatible API changes)
uint8_t	minor	Minor version number (incremented for backward-compatible functionality)
uint8_t	patch	Patch version number (incremented for backward-compatible bug fixes)
uint32_t	commit_id	Git commit identifier or build number.

7.7.3 Function Documentation

7.7.3.1 get_v2x_version()

```
prime_err_t get_v2x_version (
            v2x_version_t * args )
```

Get V2X software version information.

Retrieves the current V2X software version including major, minor, patch numbers and commit identifier.

Parameters

out	*args*	Pointer to version structure to be filled with version information

Returns

PRIME_ERR_NONE on success

PRIME_ERR_INVALID_PARAM if args is NULL

PRIME_ERROR on other errors

7.8 PRIME Cryptographic Engine

PRIME (Performance-optimized Real-time Integrated Message Encryption) API.

Modules

- PRIME Handle Management
  Handle definitions and management for PRIME services.
- PRIME Information Retrieval
  PRIME service information and capabilities.
- PRIME Service Management
  PRIME service session lifecycle management.
- PRIME Utility Definitions
  Common utility types and error codes for PRIME API.
- V2X Version Information
  V2X software version retrieval.

Data Structures

- struct status_buf
  Status buffer structure updated by firmware. More...
- struct crypto_status
  Cryptographic operation status tracking. More...
- struct prime_crypto_op
  PRIME cryptographic operation descriptor. More...
- union prime_crypto_op.__unnamed2__

Typedefs

- typedef struct status_buf status_buf_t

  Status buffer structure updated by firmware.
- typedef struct prime_crypto_op crypto_op_args_t

  PRIME cryptographic operation descriptor.

Enumerations

- enum crypto_op_type {
  CRYPTO_OP_TYPE_NONE ,
  CRYPTO_OP_TYPE_AES ,
  CRYPTO_OP_TYPE_AEAD ,
  CRYPTO_OP_TYPE_HMAC ,
  CRYPTO_OP_TYPE_HASH ,
  CRYPTO_OP_TYPE_MAX }

  Cryptographic operation types.
- enum crypto_op_status {
  CRYPTO_OP_COMPLETED ,
  CRYPTO_OP_SUBMITTED ,
  CRYPTO_OP_FIFO_FULL ,
  CRYPTO_OP_INVALID_HANDLE ,
  CRYPTO_OP_INVALID_ALGO ,
  CRYPTO_OP_MSG_BUILD_FAILED }

  Status of cryptographic operation.

Functions

- int prime_process_ops (prime_hdl_t service_hdl, crypto_op_args_t ∗∗ops, uint32_t nb_ops)

  Process cryptographic operations.

7.8.1 Detailed Description

PRIME (Performance-optimized Real-time Integrated Message Encryption) API.

This is the main interface for PRIME cryptographic operations providing:

- AES cipher operations (CBC, ECB, CTR)
- AEAD operations (GCM, CCM)
- HMAC and SHA hash operations
- Asynchronous operation processing

7.8.2 Data Structure Documentation

7.8.2.1 struct status_buf

Status buffer structure updated by firmware.

This structure contains the operation status information written by the firmware upon completion of a cryptographic operation.

Data Fields

uint8_t	status_code	Status code (0xD6 = success, other values indicate errors)
uint8_t	error_info	Additional error information for debugging.
uint16_t	reserved	Reserved for future use.

7.8.2.2 struct crypto_status

Cryptographic operation status tracking.

Contains physical and virtual addresses of the status buffer used for tracking operation completion.

Data Fields

status_buf_t *	phys_addr	Physical address of status buffer (for hardware access)
status_buf_t *	virt_addr	Virtual address of status buffer (for software access)

7.8.2.3 struct prime_crypto_op

PRIME cryptographic operation descriptor.

Main structure describing a cryptographic operation to be processed by the PRIME engine. Contains source/destination buffers, operation type, and type-specific parameters.

Data Fields

struct buf	src	Source data buffer (input for encryption/hashing)
struct buf	dst	Destination data buffer (output for encryption/hashing)
uint32_t	req_id	Request identifier for tracking (to be deleted from user interface)
struct crypto_status	crypto_status	Status tracking structure for operation completion.
enum crypto_op_status	op_status	Current status of the operation.
enum crypto_op_type	op_type	Type of cryptographic operation to perform.
union prime_crypto_op	__unnamed__	

7.8.2.4 union prime_crypto_op.__unnamed2__

Data Fields

struct aes_op_params	op_aes_args	AES cipher operation parameters.
struct aead_op_params	op_aead_args	AEAD operation parameters.
struct mac_op_params	op_mac_args	HMAC/MAC operation parameters.
struct hash_one_go_op_params	op_hash_args	Hash one go operation parameters.

7.8.3 Typedef Documentation

7.8.3.1 status_buf_t

```
typedef struct status_buf status_buf_t
```

Status buffer structure updated by firmware.

This structure contains the operation status information written by the firmware upon completion of a cryptographic operation.

7.8.3.2 crypto_op_args_t

```
typedef struct prime_crypto_op crypto_op_args_t
```

PRIME cryptographic operation descriptor.

Main structure describing a cryptographic operation to be processed by the PRIME engine. Contains source/destination buffers, operation type, and type-specific parameters.

7.8.4 Enumeration Type Documentation

7.8.4.1 crypto_op_type

```
enum crypto_op_type
```

Cryptographic operation types.

Enumeration of supported cryptographic operation types that can be processed by the PRIME engine.

Enumerator

CRYPTO_OP_TYPE_NONE	No operation or uninitialized.
CRYPTO_OP_TYPE_AES	AES cipher operation (CBC, ECB, CTR)
CRYPTO_OP_TYPE_AEAD	AEAD operation (GCM, CCM)
CRYPTO_OP_TYPE_HMAC	HMAC or SHA hash operation.
CRYPTO_OP_TYPE_HASH	Hash one go operation (standalone hash, no key required)
CRYPTO_OP_TYPE_MAX	Maximum operation type (boundary marker)

7.8.4.2 crypto_op_status

```
enum crypto_op_status
```

Status of cryptographic operation.

Indicates the current state of a cryptographic operation in the processing pipeline.

Enumerator

CRYPTO_OP_COMPLETED	Operation completed, dequeue done after receiving interrupt.
CRYPTO_OP_SUBMITTED	Operation submitted, enqueue done.
CRYPTO_OP_FIFO_FULL	Operation failed, FIFO is full.
CRYPTO_OP_INVALID_HANDLE	Operation failed, invalid handle.
CRYPTO_OP_INVALID_ALGO	Operation failed, invalid cryptographic algorithm.
CRYPTO_OP_MSG_BUILD_FAILED	Failed to build MU message.

7.8.5 Function Documentation

7.8.5.1 prime_process_ops()

```
int prime_process_ops (
            prime_hdl_t service_hdl,
            crypto_op_args_t ** ops,
            uint32_t nb_ops )
```

Process cryptographic operations.

Submit one or more cryptographic operations to the PRIME engine for processing. Operations are queued and processed asynchronously by the hardware accelerator.

Parameters

in	*service_hdl*	Handle to the PRIME service session
in,out	*ops*	Array of pointers to cryptographic operation descriptors
in	*nb_ops*	Number of operations in the array

Returns

Number of operations successfully submitted (>= 0)

Negative value on error

Index

aead_op_params, 16
aes_key_t, 16
aes_op_params, 16

buf, 29

CRYPTO_AEAD_AES_CCM
    PRIME AES Cryptographic Operations, 17
CRYPTO_AEAD_AES_GCM
    PRIME AES Cryptographic Operations, 17
crypto_aead_algorithm
    PRIME AES Cryptographic Operations, 17
CRYPTO_AUTH_OP_HMAC_GEN
    PRIME HMAC and Hash Operations, 22
CRYPTO_AUTH_OP_HMAC_VERIFY
    PRIME HMAC and Hash Operations, 22
CRYPTO_AUTH_OP_SHA_GEN
    PRIME HMAC and Hash Operations, 22
CRYPTO_AUTH_SHA_256
    PRIME HMAC and Hash Operations, 22
CRYPTO_AUTH_SHA_384
    PRIME HMAC and Hash Operations, 22
CRYPTO_AUTH_SHA_512
    PRIME HMAC and Hash Operations, 22
CRYPTO_CIPHER_AES_CBC
    PRIME AES Cryptographic Operations, 17
CRYPTO_CIPHER_AES_CTR
    PRIME AES Cryptographic Operations, 17
CRYPTO_CIPHER_AES_ECB
    PRIME AES Cryptographic Operations, 17
crypto_cipher_algorithm
    PRIME AES Cryptographic Operations, 17
CRYPTO_CIPHER_OP_DECRYPT
    PRIME AES Cryptographic Operations, 18
CRYPTO_CIPHER_OP_ENCRYPT
    PRIME AES Cryptographic Operations, 18
crypto_cipher_operation
    PRIME AES Cryptographic Operations, 17
crypto_hash_algorithm
    PRIME HMAC and Hash Operations, 22
CRYPTO_HASH_SHA2_256
    PRIME HMAC and Hash Operations, 23
CRYPTO_HASH_SHA2_384
    PRIME HMAC and Hash Operations, 23
CRYPTO_HASH_SHA2_512
    PRIME HMAC and Hash Operations, 23
crypto_mac_operation
    PRIME HMAC and Hash Operations, 22
crypto_op_args_t
    PRIME Cryptographic Engine, 34
CRYPTO_OP_COMPLETED
    PRIME Cryptographic Engine, 35
CRYPTO_OP_FIFO_FULL
    PRIME Cryptographic Engine, 35
CRYPTO_OP_INVALID_ALGO
    PRIME Cryptographic Engine, 35
CRYPTO_OP_INVALID_HANDLE
    PRIME Cryptographic Engine, 35
CRYPTO_OP_MSG_BUILD_FAILED
    PRIME Cryptographic Engine, 35
crypto_op_status
    PRIME Cryptographic Engine, 34
CRYPTO_OP_SUBMITTED
    PRIME Cryptographic Engine, 35
crypto_op_type
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_AEAD
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_AES
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_HASH
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_HMAC
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_MAX
    PRIME Cryptographic Engine, 34
CRYPTO_OP_TYPE_NONE
    PRIME Cryptographic Engine, 34
crypto_sha_algorithm
    PRIME HMAC and Hash Operations, 22
crypto_status, 33

get_v2x_version
    V2X Version Information, 31

hash_one_go_args_t, 21
hash_one_go_op_params, 21
hmac_key_t, 20

mac_op_params, 21
MAX_PRIME_KEY_SLOT_SUPPORTED
    PRIME Service Management, 27

open_service_args_t, 26

PRIME AES Cryptographic Operations, 15
    CRYPTO_AEAD_AES_CCM, 17
    CRYPTO_AEAD_AES_GCM, 17
    crypto_aead_algorithm, 17
    CRYPTO_CIPHER_AES_CBC, 17
    CRYPTO_CIPHER_AES_CTR, 17

CRYPTO_CIPHER_AES_ECB, 17
crypto_cipher_algorithm, 17
CRYPTO_CIPHER_OP_DECRYPT, 18
CRYPTO_CIPHER_OP_ENCRYPT, 18
crypto_cipher_operation, 17
prime_cipher_init, 18
PRIME Cryptographic Engine, 31
crypto_op_args_t, 34
CRYPTO_OP_COMPLETED, 35
CRYPTO_OP_FIFO_FULL, 35
CRYPTO_OP_INVALID_ALGO, 35
CRYPTO_OP_INVALID_HANDLE, 35
CRYPTO_OP_MSG_BUILD_FAILED, 35
crypto_op_status, 34
CRYPTO_OP_SUBMITTED, 35
crypto_op_type, 34
CRYPTO_OP_TYPE_AEAD, 34
CRYPTO_OP_TYPE_AES, 34
CRYPTO_OP_TYPE_HASH, 34
CRYPTO_OP_TYPE_HMAC, 34
CRYPTO_OP_TYPE_MAX, 34
CRYPTO_OP_TYPE_NONE, 34
prime_process_ops, 35
status_buf_t, 34
PRIME Handle Management, 18
PRIME_HANDLE_NONE, 19
prime_hdl_t, 19
PRIME HMAC and Hash Operations, 19
CRYPTO_AUTH_OP_HMAC_GEN, 22
CRYPTO_AUTH_OP_HMAC_VERIFY, 22
CRYPTO_AUTH_OP_SHA_GEN, 22
CRYPTO_AUTH_SHA_256, 22
CRYPTO_AUTH_SHA_384, 22
CRYPTO_AUTH_SHA_512, 22
crypto_hash_algorithm, 22
CRYPTO_HASH_SHA2_256, 23
CRYPTO_HASH_SHA2_384, 23
CRYPTO_HASH_SHA2_512, 23
crypto_mac_operation, 22
crypto_sha_algorithm, 22
prime_hash_one_go, 23
prime_mac_init, 23
PRIME Information Retrieval, 24
prime_get_info, 25
PRIME Service Management, 25
MAX_PRIME_KEY_SLOT_SUPPORTED, 27
prime_close_service, 28
prime_open_service, 27
service_type, 27
SERVICE_TYPE_ALL, 27
SERVICE_TYPE_NONE, 27
PRIME Utility Definitions, 28
PRIME_ERR_INVALID_PARAM, 30
PRIME_ERR_MEMORY_ALLOC, 30
PRIME_ERR_NONE, 30
PRIME_ERR_SERVICE_CLOSE, 30
PRIME_ERR_SERVICE_OPEN, 30
prime_err_t, 29
PRIME_ERROR, 30
prime_cipher_init
PRIME AES Cryptographic Operations, 18
prime_close_service
PRIME Service Management, 28
prime_crypto_op, 33
prime_crypto_op.__unnamed2__, 33
PRIME_ERR_INVALID_PARAM
PRIME Utility Definitions, 30
PRIME_ERR_MEMORY_ALLOC
PRIME Utility Definitions, 30
PRIME_ERR_NONE
PRIME Utility Definitions, 30
PRIME_ERR_SERVICE_CLOSE
PRIME Utility Definitions, 30
PRIME_ERR_SERVICE_OPEN
PRIME Utility Definitions, 30
prime_err_t
PRIME Utility Definitions, 29
PRIME_ERROR
PRIME Utility Definitions, 30
prime_get_info
PRIME Information Retrieval, 25
PRIME_HANDLE_NONE
PRIME Handle Management, 19
prime_hash_one_go
PRIME HMAC and Hash Operations, 23
prime_hdl_t
PRIME Handle Management, 19
prime_info_t, 24
prime_mac_init
PRIME HMAC and Hash Operations, 23
prime_open_service
PRIME Service Management, 27
prime_process_ops
PRIME Cryptographic Engine, 35

service_type
PRIME Service Management, 27
SERVICE_TYPE_ALL
PRIME Service Management, 27
SERVICE_TYPE_NONE
PRIME Service Management, 27
status_buf, 32
status_buf_t
PRIME Cryptographic Engine, 34

V2X Version Information, 30
get_v2x_version, 31
v2x_version_t, 30